

Non-Webhook Fixes — Production Audit Follow-Up

A build-ready plan for every remaining finding in the 1 Aug 2026 production audit, grouped by priority with file-level scope, an architecture-preserving approach, and time estimates.

Explicitly out of scope: the Calendly webhook receiver, signature verification, and every other webhook-dependent finding are excluded from this plan — no code in this document touches that path. Everything below assumes the current architecture stays as-is: same Flask blueprint structure, same React component/hook patterns, no new services or infrastructure unless called out.

00 Overview

TOTAL TASKS	EST. TOTAL EFFORT	REQUIRED BEFORE LAUNCH	CAN WAIT POST-LAUNCH	MARKED OPTIONAL
34	~61–65 hrs	18 tasks	16 tasks	3 tasks

Every task below reuses an existing component, hook, service, or utility already in the codebase — nothing here introduces a new framework, library, or architectural layer unless explicitly flagged as OPTIONAL. Time estimates assume a developer already familiar with this codebase's conventions.

Block launch on these. None require new infrastructure — all three are self-contained fixes to existing files.

CRITICAL

~3-4 HRS

Optimize oversized images, remove unused assets

SCOPE	Re-export <code>Home/components/HomeHero.jsx</code> 's 24.6MB hero image and the shared 9.88MB About/Blog hero background as WebP at their actual display size (target <200KB each). Delete <code>public/partners/amit.png</code> (33MB, already unreferenced — <code>data/partners.js</code> uses <code>amit.webp</code>). Re-encode or replace <code>public/article/Akshit Arora-.mp4</code> (59MB) to under 10MB with <code>ffmpeg</code> .
APPROACH	Follow the existing convention already used for the partner photos (WebP export, same directory structure). No component logic changes — swap the <code>src</code> import only. Keep the same <code></code> / <code><motion.img></code> markup and add <code>loading="lazy"</code> only where the element isn't already above-the-fold-critical.
UI IMPACT	None — same visual result at a fraction of the file size, per the "do not change UI or design" constraint.

CRITICAL

~3-4 HRS

Fix placeholder page title, add missing SEO metadata

SCOPE	Replace the literal <code>"profile-project"</code> in <code>frontend/index.html</code> with the firm's real name. Add the existing <code><Seo></code> component to the ~12 routes that don't have it yet: Home, About, Contact, Career, Blog listing, and every hand-built Service page.
APPROACH	Reuse the exact <code><Seo title=... description=... canonicalPath=... /></code> pattern already working on the 4 routes that have it (Appointment, InsightArticle, Insights). No new component needed — just wire it in and write per-page title/description copy.

CRITICAL

~1-2 HRS

Pin the config-default footgun

SCOPE	<code>backend/config/settings.py:157-159</code> — stop <code>get_config()</code> from silently falling back to <code>DevelopmentConfig</code> when <code>FLASK_ENV</code> is unset. <code>backend/run.py:6</code> — stop <code>debug=True</code> , <code>host="0.0.0.0"</code> from being reachable outside local development.
APPROACH	Same config-class structure, just change the fallback branch to raise/default to production-safe values, and gate <code>run.py</code> 's arguments behind <code>app.config["DEBUG"]</code> (which is already read correctly elsewhere). No new config system — the existing <code>DevelopmentConfig</code> / <code>ProductionConfig</code> classes are correct, only the selection logic needs to change.

Fix before real client traffic arrives.

HIGH

~2 HRS

Add a branded 404 page and router error boundary

SCOPE `frontend/src/website/routes/AppRoutes.jsx:20-42` — add a `{ path: "**", element: <NotFound /> }` route and an `errorElement` on the root layout route.

APPROACH Build `NotFound` using the site's existing page-shell/layout components and typography — same header/footer as every other page, not a standalone design. Reuse whatever "empty state" visual pattern the admin panel's `ErrorState` component already establishes, adapted to the public site's styling.

HIGH

~3-4 HRS

Make the newsletter send non-blocking (without adding new infrastructure)

SCOPE `backend/app/services/newsletter_service.py:140-189` currently loops and sends synchronously inside the request.

APPROACH (ARCHITECTURE-PRESERVING) Do **not** introduce Celery/RQ/Redis — that's a new infrastructure dependency and out of scope per your constraints. Instead, run the send loop in a `concurrent.futures.ThreadPoolExecutor` (stdlib, no new dependency) kicked off from the route handler, return `202` immediately with the campaign id, and have the thread update the existing `NewsletterCampaign` row's status/sent-count as it progresses so a crash mid-send leaves a resumable record instead of silence. The admin UI already has a pattern for refetch-driven progress polling (`useAsyncData` + `refetch`) that a "sending..." status view can reuse.

OPTIONAL UPGRADE **OPTIONAL** A real task queue (Celery/RQ + Redis) is the more scalable long-term fix once subscriber volume grows past what a thread pool comfortably handles — not needed for initial launch.

HIGH

~2-3 HRS

Add route-level code splitting

SCOPE `frontend/src/website/routes/AppRoutes.jsx:4-17` — every page is statically imported into one bundle.

APPROACH Wrap each route's page import in `React.lazy()` ; wrap the router's `<Outlet/>` (or each lazy element) in `<Suspense fallback={...}>` using an existing skeleton/loading component already used elsewhere on the site, so the loading state looks consistent with the rest of the app rather than a generic spinner. No route paths or page components change — only how they're imported.

UI IMPACT None visible on fast connections; on slow connections, a brief skeleton instead of a longer blank wait before the first paint.

HIGH

~3-4 HRS

Fix N+1 queries on public list endpoints, paginate the blog list

SCOPE `backend/app/blueprints/blog/routes.py:8-35` (category/author/tags/key_takeaways/faqs each lazy-loaded per row, plus an unpaginated `list_posts`); same per-row `Media.query.get()` pattern in `team/routes.py`, `testimonials/routes.py`, `services/routes.py`, `admin/service_routes.py`.

APPROACH Add `selectinload` / `joinedload` options to the existing queries — no query-building library change, this is a stock SQLAlchemy feature already imported elsewhere in the ORM layer. Wire the blog list into the existing `paginate_query` utility already used by every admin list endpoint, rather than building new pagination logic.

HIGH

~4-6 HRS

RECOMMENDED, NOT BLOCKING

Stand up a minimal automated test suite + CI

SCOPE No tests exist anywhere in the repo today. Start narrow rather than attempting full coverage.

APPROACH Add `pytest` + a test config that reuses the existing Flask app factory (`create_app()`) against a throwaway SQLite or test-MySQL DB. Cover: (1) the refresh-token rotation/reuse-detection logic — the single most failure-sensitive path in the app, (2) one contact/career idempotency test once \$High's duplicate-submission fix lands, (3) one admin CRUD round-trip using the existing `createResourceApi` / `admin_crud` factory pattern as a smoke test. Add a GitHub Actions workflow that runs `pytest` and `eslint` . on every PR — both commands already exist as scripts, CI just needs to invoke them.

NOTE Doesn't fix a live defect, so it's safe to run in parallel with launch prep rather than gating it — see the launch-split section below.

HIGH

~3-4 HRS DEV + LEGAL REVIEW

Build real Privacy Policy and Terms pages, fix the broken link

SCOPE The Contact form's mandatory consent checkbox links to `href="/"` (`ContactForm.jsx:339`); no privacy policy or terms page exists.

APPROACH Add `/privacy-policy` and `/terms` as new routes using the site's existing simple content-page layout (the same shell used by static pages like Insights articles). Update the Contact form link, and add both links to the site footer and the Career application form for consistency. **Dev builds the pages and wiring; the firm/legal needs to supply the actual policy text** — flag this dependency early since it can block the "before launch" timeline if content isn't ready.

HIGH

~2-3 HRS

Add duplicate-submission protection to Contact and Career forms

SCOPE

`backend/app/services/contact_service.py:10-36` and `career_service.py:11-71` create a row unconditionally on every request — a double-click or client retry creates duplicate enquiries/applications and duplicate emails.

APPROACH

Mirror the idempotent-upsert style already used for the appointment embed-intake handler in `appointment_service.py` (select-first on a short-lived dedupe key, commit, fall back to a re-fetch on an `IntegrityError` race) — *this references only the existing embed-form-intake code, not the webhook path, which stays untouched*. A practical dedupe key here: same email + message hash within a short window (e.g. 2 minutes).

Cheap wins mixed with a few that need a small migration — none require architectural changes.

MEDIUM

~15-30 MIN

Untrack `frontend/.env` from git

SCOPE `frontend/.env` is currently committed; only `backend/.env` is gitignored.

APPROACH Add the same rule already working for the backend to the frontend's `.gitignore`, then `git rm --cached frontend/.env`. Keep `frontend/.env.example` for onboarding, matching the backend's existing pattern exactly.

MEDIUM

~30 MIN - 2.5 HRS

Restrict or sanitize SVG uploads

SCOPE `backend/app/utils/file_utils.py:24-31` allows `.svg / image/svg+xml` with no content sanitization.

APPROACH Fastest fix (~30 min): drop `.svg` from `ALLOWED_IMAGE_EXTENSIONS / ALLOWED_IMAGE_MIME_TYPES` if admins don't specifically need SVG uploads. If they do, add a sanitization step (e.g. `bleach`) in the same validation function that already does MIME-sniffing — no change to the upload flow's shape, just a stricter check inside the existing validator.

MEDIUM

~1 HR

Wrap admin-only routes in the existing `RoleGuard`

SCOPE `frontend/src/admin/routes/AdminRoutes.jsx:56-88` — Users, Audit Log, Security, and Settings have no client-side role gate. (Backend already enforces this correctly on every endpoint — this is a UX fix, not a security fix.)

APPROACH `RoleGuard` already exists in `auth/RoleGuard.jsx` and is unused — wrap the relevant route subtree with it, same component, zero new code beyond the wiring.

MEDIUM

~30 MIN

Fix the Security page's broken retry

SCOPE `pages/Security/Security.jsx:22,41-43` — retry only re-fetches sessions, never failed-logins.

APPROACH Destructure `refetch: refetchFailedLogins` from the existing `useAsyncData` call already used for that data, and call both refetchers from the retry handler.

MEDIUM

~2-3 HRS CHECK BACKEND FIELD FIRST

Make BlogPosts/FirmStats delete permissions consistent

SCOPE `pages/BlogPosts/BlogPosts.jsx:104-111` , `pages/FirmStats/FirmStats.jsx:73` allow any authenticated user a single-click permanent delete, unlike Team/Awards/Testimonials/JobOpenings.

APPROACH Apply the same two-step soft-delete + admin-only gate already built for Team/Awards — reuse that pattern's frontend component structure directly. **Check first** whether `blog_posts` / `firm_stats` already have an `is_active` -style column like the other tables; if not, this needs a small additive migration (nullable boolean, default true) before the frontend change, following the same safe-backfill style already used in this project's migration history.

MEDIUM

~30 MIN

Actually enforce the `required` prop on admin form inputs

SCOPE `components/form/Field.jsx:25-60` — `required` is destructured for the label asterisk but never spread onto the actual input/textarea/select.

APPROACH One-line fix per field type in the shared component — cascades correctly to every admin form using it, no per-page changes needed.

MEDIUM

~2-3 HRS

Add a focus trap to modals/dialogs

SCOPE `components/ConfirmDialog.jsx:19-27` , `components/Drawer.jsx:7-18` — Escape and initial focus work, but Tab can move behind the overlay.

APPROACH Add one small shared `useFocusTrap` hook (stdlib DOM APIs only, no new dependency) alongside the project's existing `hooks/` folder, and apply it to both components — a single reusable addition rather than two one-off fixes.

MEDIUM

~30 MIN

Reconcile the `services` model with its existing indexes

SCOPE `backend/app/models/service.py:34,53,69` — three FK columns have indexes in the DB (from a prior migration) that the model doesn't declare, risking a future autogenerate accidentally dropping them.

APPROACH Add `index=True` to those three columns in the model. No migration needed — the indexes already exist in the database.

MEDIUM

~1 HR

Add a composite index on `AuditLog(entity_type, entity_id)`

SCOPE `backend/app/models/admin.py:29-30` — the natural "history for this record" query has no composite index to use.

APPROACH One additive Alembic migration, following the same style as the project's existing 7 migrations (linear, no destructive changes). Drop the now-redundant single-column `entity_type` index in the same migration.

MEDIUM

~2-3 HRS

Audit naive vs. timezone-aware datetime handling

SCOPE `backend/app/models/mixins.py:10-14` — `DateTime(timezone=True)` doesn't actually persist tz info under MySQL; the appointment reminder job depends on comparing these values correctly.

APPROACH Standardize on naive-UTC storage and re-attach `tzinfo=UTC` consistently wherever a value is read back out, matching whatever convention the reminder-scheduling code already leans toward. This is a data-correctness fix, not a webhook change — no Calendly credentials involved.

MEDIUM

~1-2 HRS

Add error monitoring

SCOPE No Sentry (or equivalent) integration exists — all error visibility is stdout logging.

APPROACH Initialize the Sentry SDK inside the existing `create_app()` factory, gated to `ProductionConfig` only — a few lines, no restructuring of the existing logging middleware.

MEDIUM

~2 HRS

Unify the 400 vs 422 validation status codes

SCOPE Public blueprints return 400 on validation failure, admin blueprints return 422, for the identical response shape.

APPROACH Standardize on 422 (more semantically correct) across the public controllers — `contact/controller.py:12`, `appointments/controller.py:22` (non-webhook validation only), `career/controller.py:12` — to match what admin already does. Pure status-code change, no response-shape or client contract change beyond that.

MEDIUM

~2 HRS

Add `robots.txt`, `sitemap.xml`, and an analytics tag

SCOPE Neither file exists in `frontend/public/`; no analytics script in `index.html`.

APPROACH Since the route list is small and mostly static, a hand-written `sitemap.xml` and a standard `robots.txt` in `public/` is enough — no build-time generator needed. Add the analytics snippet directly to `index.html` the same way the favicon link is already declared there.

MEDIUM

~1 HR

OPTIONAL

Add HTTP caching headers for static media

SCOPE Uploaded media served from `/media/<filename>` has no cache-control headers.

APPROACH Add a long `Cache-Control: public, max-age=...` header in the existing static-file-serving route, since filenames are already UUID-based (content-addressed in effect) — safe to cache aggressively without invalidation logic.

MEDIUM

~1-2 DAYS

OPTIONAL

Admin two-factor authentication

SCOPE No second factor exists for admin login today.

APPROACH TOTP-based (Google Authenticator-compatible), added as an optional field on the existing `Admin` model plus a verify-step in the existing login flow — additive to the current auth service, not a rewrite of it. This is a genuine feature addition, not a defect fix, so it's marked optional and shouldn't block launch.

04 Low

Polish. Batch these into a single cleanup pass rather than context-switching individually.

TASK	FILE(S)	APPROACH	EST.
Standardize decorative-image alt / aria-hidden usage	AboutHero.jsx , ContactHero.jsx , Service Hero.jsx files	Match the already-correct pattern used in BlogHero.jsx / LifeHero.jsx	~1 hr
Delete unused GenericServicePage.jsx	pages/Services/GenericServicePage.jsx	Confirmed unreferenced — remove outright	~15 min
Import TABLE_ARGS instead of redefining it	backend/app/models/auth.py:17	from app.models.admin import TABLE_ARGS	~15 min
Change FirmStat.value to Integer	backend/app/models/firm.py:42-43	Additive migration + cast existing values + update the admin form's validator	~1–2 hrs
Add IntegrityError fallback on uniqueness pre-checks	validations/content_validator.py:22-28 , utils/admin_crud.py:66-76	Catch-and-translate to the existing 422 field-error shape	~1–2 hrs
Clean up orphaned files on a failed DB commit	career_service.py:20-47 , admin/media_routes.py:82-98	Wrap in try/except, delete the just-saved file on failure	~1–2 hrs
Give TagMultiSelect 's input a real label	components/TagMultiSelect.jsx:12,25-31	Match the id / htmlFor pattern used by form/Field.jsx	~30 min
Fix the stray useAsyncData(fetcher, []) argument	pages/Dashboard/Dashboard.jsx:34	Remove the unused second argument	~15 min
Verify production MySQL is ≥8.0.16	Infra check, not code	Confirm SELECT VERSION(); on the prod host so the Testimonial.rating CHECK constraint actually enforces	~15 min
Extract shared form/animation boilerplate OPTIONAL	ContactForm.jsx , BookingSection.jsx , ApplyNow.jsx	Pull EASE / fadeUp / inputBaseClasses /error-message markup into one shared module — refactor only, no behavior change	~3–4 hrs
Admin session-expiry warning toast OPTIONAL	auth/tokenStore.js refresh scheduler	Reuse the existing ToastProvider to warn ~60s before a forced logout	~2 hrs

Consolidated Checklist

Every task above, flattened and ordered highest to lowest priority.

CRITICAL

- ☐ **Optimize/replace oversized images** — hero, About/Blog background, delete unused public/ assets 3-4H
- ☐ **Fix page title + add per-route SEO metadata** 3-4H
- ☐ **Pin config defaults** (FLASK_ENV fallback + run.py debug/host) 1-2H

HIGH

- ☐ **Untrack frontend/.env** from git 15-30M
- ☐ **Branded 404 page + router error boundary** 2H
- ☐ **Contact/Career duplicate-submission protection** 2-3H
- ☐ **Privacy Policy + Terms pages, fix broken link** 3-4H+LEGAL
- ☐ **Restrict/sanitize SVG uploads** 30M-2.5H
- ☐ Non-blocking newsletter send (thread pool, no new infra) 3-4H
- ☐ Route-level code splitting 2-3H
- ☐ Fix N+1 queries + paginate blog list 3-4H
- ☐ Minimal test suite + CI 4-6H

MEDIUM

- ☐ RoleGuard on admin-only routes 1H
- ☐ Fix Security.jsx broken retry 30M
- ☐ Enforce `required` on admin inputs 30M
- ☐ Reconcile services model indexes 30M
- ☐ Composite index on AuditLog 1H
- ☐ BlogPosts/FirmStats delete-permission consistency 2-3H

☐ Focus trap on dialogs/drawer	2-3H
☐ Datetime naive/aware audit	2-3H
☐ Add Sentry error monitoring	1-2H
☐ Unify 400 vs 422 status codes	2H
☐ robots.txt, sitemap.xml, analytics tag	2H
☐ HTTP caching headers for media OPTIONAL	1H
☐ Admin 2FA OPTIONAL	1-2D

LOW

☐ Alt-text/aria-hidden consistency pass	1H
☐ Delete GenericServicePage.jsx	15M
☐ Import TABLE_ARGS in auth.py	15M
☐ FirmStat.value → Integer	1-2H
☐ IntegrityError fallback on uniqueness checks	1-2H
☐ Clean up orphaned files on commit failure	1-2H
☐ TagMultiSelect label association	30M
☐ Fix stray useAsyncData argument	15M
☐ Verify prod MySQL ≥8.0.16	15M
☐ Extract shared form/animation boilerplate OPTIONAL	3-4H
☐ Session-expiry warning toast OPTIONAL	2H

Required vs. Can Wait

REQUIRED BEFORE LAUNCH

- Oversized image optimization + asset cleanup
- Page title + SEO metadata
- Config-default fix (FLASK_ENV / run.py)
- Untrack frontend/.env
- 404 page + error boundary
- Contact/Career duplicate protection
- Privacy Policy + Terms pages (content permitting)
- SVG upload restriction
- RoleGuard on admin routes
- Security.jsx retry fix
- Required-field enforcement in admin forms
- BlogPosts/FirmStats delete-permission fix
- Verify prod MySQL version

SAFE TO SCHEDULE POST-LAUNCH

- Non-blocking newsletter send (fine until first large campaign)
- Route-level code splitting (perf, not correctness)
- N+1 query fixes (matters more as content volume grows)
- Test suite + CI
- Focus trap, datetime audit, status-code unification
- Sentry, robots.txt/sitemap/analytics — do within week 1
- services/AuditLog index reconciliation
- All Low-tier polish items
- Both OPTIONAL items (2FA, session-expiry toast)
- Optional shared-boilerplate refactor

Recommended Execution Order

Phased so the cheapest, highest-leverage fixes land first and legal-content dependencies (Privacy Policy copy) get kicked off early since they're off the critical dev path.

Phase 1 — Quick wins

~2-3 hrs

- Pin config defaults (FLASK_ENV / run.py)
- Untrack frontend/.env
- Restrict SVG uploads (quick version)
- Enforce `required` prop in `Field.jsx`
- Fix Security.jsx retry
- Reconcile services model indexes
- *In parallel:* kick off Privacy Policy/Terms copy with the firm's legal side — this has the longest lead time and isn't on the dev critical path

Phase 2 — Critical assets & SEO

~6-8 hrs

- Image optimization + unused-asset cleanup
- Page title fix + per-route SEO metadata
- robots.txt / sitemap.xml / analytics tag

Phase 3 — Correctness & data integrity

~8-10 hrs

- Contact/Career duplicate-submission protection
- 404 page + router error boundary
- RoleGuard on admin routes
- BlogPosts/FirmStats delete-permission fix
- Composite AuditLog index

Phase 4 — Performance & scale

~9-11 hrs

- Route-level code splitting
- N+1 query fixes + blog pagination
- Non-blocking newsletter send

Phase 5 — Launch gate

—

- Wire in Privacy Policy/Terms content once legal delivers copy
- Verify prod MySQL version
- Final pass through the Required column in §6
- 🚀 Launch

Phase 6 — First post-launch week

~6-8 hrs

- Sentry integration
- Focus trap on dialogs
- Datetime naive/aware audit
- 400 vs 422 status-code unification
- Start the minimal test suite + CI

Phase 7 — Backlog

~10-12 hrs

- All Low-tier polish items
- OPTIONAL: admin 2FA
- OPTIONAL: session-expiry toast
- OPTIONAL: shared form/animation boilerplate extraction
- OPTIONAL: HTTP caching headers for media

Derived strictly from the 1 Aug 2026 production audit • All Calendly webhook findings excluded per instruction •
No architecture changes, no new infrastructure introduced.
